

Generative Bayesian Classifiers, Heteroscedastic Gaussian Densities, and Non-Linear Decision Geometries

A Unified Mathematical, Information-Theoretic, and Physical Foundation of
Quadratic Discriminant Analysis (QDA)

Contents

1	The Generative Learning Paradigm and Heteroscedasticity	2
1.1	The Heteroscedastic Assumption: Unconstrained Variance	2
2	Derivation of the Quadratic Discriminant Function	2
2.1	Matrix Expansion and Non-Linear Algebra	3
3	Geometry of Quadratic Hyper-Quadric Decision Boundaries	3
4	Statistical Complexity and Parameter Capacity	4
4.1	Parameter Counting Comparison	4
5	Regularized Discriminant Analysis (RDA): A Continuous Spectrum	4
5.1	Step 1: Interpolating Between QDA and LDA	4
5.2	Step 2: Shrinkage Toward Isotropic Variance	5
6	Physical Analogies: Anisotropic Potential Wells and Entropic Basins	5
6.1	Anisotropic Harmonic Potential Wells	5
6.2	Entropic Penalty and Basin Volume	5
7	Maximum Likelihood Estimation and Numerical Computations	6
7.1	Parameter Estimation	6
7.2	Numerical Stability via Cholesky Decomposition	6
8	Production-Grade Vectorized NumPy Implementation	6

1 The Generative Learning Paradigm and Heteroscedasticity

Generative learning algorithms perform classification by modeling the joint probability distribution $P(\mathbf{X}, Y) = P(\mathbf{X} | Y)P(Y)$. By fitting class-conditional data densities $f_k(\mathbf{x}) = P(\mathbf{X} = \mathbf{x} | Y = k)$ alongside categorical class priors $\pi_k = P(Y = k)$, Bayes' Theorem determines posterior class membership:

$$P(Y = k | \mathbf{X} = \mathbf{x}) = \frac{f_k(\mathbf{x})\pi_k}{\sum_{l=1}^K f_l(\mathbf{x})\pi_l} \quad (1)$$

Definition 1.1 (Generative Gaussian Framework). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be an i.i.d. dataset where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{1, 2, \dots, K\}$. We assume that each class density $f_k(\mathbf{x})$ follows a d -dimensional Multivariate Gaussian distribution:

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\mathbf{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right) \quad (2)$$

where $\boldsymbol{\mu}_k \in \mathbb{R}^d$ is the mean vector of class k , and $\mathbf{\Sigma}_k \in \mathbb{R}^{d \times d}$ is the symmetric positive-definite covariance matrix of class k .

1.1 The Heteroscedastic Assumption: Unconstrained Variance

Linear Discriminant Analysis (LDA) enforces homoscedasticity ($\mathbf{\Sigma}_1 = \mathbf{\Sigma}_2 = \dots = \mathbf{\Sigma}_K = \mathbf{\Sigma}$), forcing all classes to share identical covariance structures.

In **Quadratic Discriminant Analysis (QDA)**, we relax this assumption to allow **Heteroscedasticity**:

$$\mathbf{\Sigma}_k \neq \mathbf{\Sigma}_l \quad \text{for } k \neq l \quad (3)$$

Each class possesses an independent, unconstrained covariance matrix $\mathbf{\Sigma}_k$. This allows class distributions to exhibit distinct spatial orientations, directional variances, and feature correlation structures in $\mathbb{R}^d$.

2 Derivation of the Quadratic Discriminant Function

According to Bayes Decision Theory, optimal decision boundaries minimize classification error by assigning an observation $\mathbf{x}$ to the class maximizing log-posterior probability:

$$\hat{y}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} [\log f_k(\mathbf{x}) + \log \pi_k] \quad (4)$$

Substituting the heteroscedastic Gaussian density $f_k(\mathbf{x})$ into the log-posterior objective:

$$\begin{aligned} \log f_k(\mathbf{x}) + \log \pi_k &= \log \left(\frac{1}{(2\pi)^{d/2}|\mathbf{\Sigma}_k|^{1/2}} \exp \left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) \right) \right) + \log \pi_k \\ &= -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\mathbf{\Sigma}_k| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k \end{aligned} \quad (5)$$

Dropping the additive constant $-\frac{d}{2} \log(2\pi)$ yields the **Quadratic Discriminant Function**:

Definition 2.1 (QDA Discriminant Function).

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \log |\mathbf{\Sigma}_k| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \mathbf{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k \quad (6)$$

2.1 Matrix Expansion and Non-Linear Algebra

Expanding the quadratic vector-matrix-vector product:

$$(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \mathbf{x}^T \boldsymbol{\Sigma}_k^{-1} \mathbf{x} - 2\mathbf{x}^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k + \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k \quad (7)$$

Substituting this expansion into $\delta_k(\mathbf{x})$:

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \mathbf{x}^T \boldsymbol{\Sigma}_k^{-1} \mathbf{x} + \mathbf{x}^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| + \log \pi_k \quad (8)$$

Unlike LDA, the quadratic term $-\frac{1}{2} \mathbf{x}^T \boldsymbol{\Sigma}_k^{-1} \mathbf{x}$ depends explicitly on class index k because $\boldsymbol{\Sigma}_k^{-1}$ varies across classes. Thus, it cannot be dropped as a constant.

We express $\delta_k(\mathbf{x})$ in general matrix form as:

$$\delta_k(\mathbf{x}) = \mathbf{x}^T \mathbf{A}_k \mathbf{x} + \mathbf{b}_k^T \mathbf{x} + c_k \quad (9)$$

where:

$$\mathbf{A}_k = -\frac{1}{2} \boldsymbol{\Sigma}_k^{-1}, \quad \mathbf{b}_k = \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k, \quad c_k = -\frac{1}{2} \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| + \log \pi_k \quad (10)$$

3 Geometry of Quadratic Hyper-Quadric Decision Boundaries

The decision boundary separating class k from class l is defined by $\delta_k(\mathbf{x}) = \delta_l(\mathbf{x})$:

$$\mathbf{x}^T \mathbf{A}_k \mathbf{x} + \mathbf{b}_k^T \mathbf{x} + c_k = \mathbf{x}^T \mathbf{A}_l \mathbf{x} + \mathbf{b}_l^T \mathbf{x} + c_l \quad (11)$$

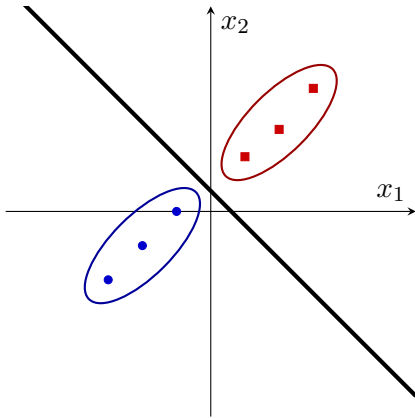
Rearranging terms yields a full second-degree polynomial matrix equation:

$$\mathbf{x}^T (\boldsymbol{\Sigma}_l^{-1} - \boldsymbol{\Sigma}_k^{-1}) \mathbf{x} + 2\mathbf{x}^T (\boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k - \boldsymbol{\Sigma}_l^{-1} \boldsymbol{\mu}_l) + M_{kl} = 0 \quad (12)$$

where $M_{kl} = \boldsymbol{\mu}_l^T \boldsymbol{\Sigma}_l^{-1} \boldsymbol{\mu}_l - \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k + \log |\boldsymbol{\Sigma}_l| - \log |\boldsymbol{\Sigma}_k| + 2 \log \left(\frac{\pi_k}{\pi_l} \right)$.

Theorem 3.1 (Hyper-Quadric Geometry). The locus of points satisfying $\delta_k(\mathbf{x}) = \delta_l(\mathbf{x})$ forms a d -dimensional **Hyper-Quadric Surface** (such as ellipsoids, hyperboloids, paraboloids, or intersecting hyperplanes).

LDA: Homoscedastic ($\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_2$)



QDA: Heteroscedastic ($\boldsymbol{\Sigma}_1 \neq \boldsymbol{\Sigma}_2$)

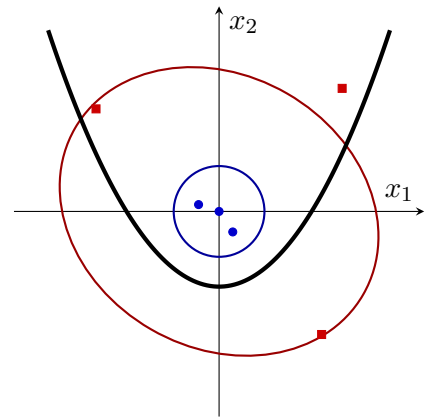


Figure 1: Homoscedastic LDA produces linear decision hyperplanes. Heteroscedastic QDA forms curved quadratic boundaries due to differing class covariance structures.

4 Statistical Complexity and Parameter Capacity

The modeling flexibility of QDA comes at the cost of significantly higher parameter complexity compared to LDA.

4.1 Parameter Counting Comparison

Let d be the feature dimension and K be the number of classes:

1. Linear Discriminant Analysis (LDA)

- Class Means: $K \times d$ parameters.
- Shared Covariance Matrix: $1 \times \frac{d(d+1)}{2}$ unique parameters.
- Class Priors: $K - 1$ independent parameters.
- ****Total Complexity:**** $\mathcal{O}\left(Kd + \frac{d^2}{2}\right)$ parameters.

2. Quadratic Discriminant Analysis (QDA)

- Class Means: $K \times d$ parameters.
- Class Covariance Matrices: $K \times \frac{d(d+1)}{2}$ unique parameters.
- Class Priors: $K - 1$ independent parameters.
- ****Total Complexity:**** $\mathcal{O}\left(K \cdot \frac{d^2}{2} + Kd\right)$ parameters.

Model	Covariance Structure	Parameter Growth	Bias / Variance Profile
LDA	Pooled shared (Σ)	Quadratic in d , linear in K	High Bias, Low Variance
QDA	Independent per class (Σ_k)	Quadratic in both d and K	Low Bias, High Variance

Table 1: Structural trade-offs between LDA and QDA models.

When the sample size per class N_k is small relative to feature dimension d ($N_k < d$), estimating $\frac{d(d+1)}{2}$ parameters per covariance matrix introduces high variance, leading to estimation error and overfitting.

5 Regularized Discriminant Analysis (RDA): A Continuous Spectrum

To bridge the gap between high-bias LDA and high-variance QDA, Friedman (1989) proposed ****Regularized Discriminant Analysis (RDA)****. RDA introduces two regularization hyperparameters $\alpha, \gamma \in [0, 1]$ to continuously interpolate between QDA, LDA, and Ridge-like isotropic estimators.

5.1 Step 1: Interpolating Between QDA and LDA

First, each individual sample covariance matrix $\hat{\Sigma}_k$ is shrunk toward the pooled covariance matrix $\hat{\Sigma}_{\text{pooled}}$ using parameter $\alpha \in [0, 1]$:

$$\hat{\Sigma}_k(\alpha) = (1 - \alpha)\hat{\Sigma}_k + \alpha\hat{\Sigma}_{\text{pooled}} \quad (13)$$

- $\alpha = 0$: Pure QDA (separate covariance per class).
- $\alpha = 1$: Pure LDA (shared pooled covariance matrix).

5.2 Step 2: Shrinkage Toward Isotropic Variance

Next, the resulting covariance matrix is shrunk toward a spherical matrix using parameter $\gamma \in [0, 1]$:

$$\hat{\Sigma}_k(\alpha, \gamma) = (1 - \gamma)\hat{\Sigma}_k(\alpha) + \gamma \left(\frac{1}{d} \text{Tr}(\hat{\Sigma}_k(\alpha)) \right) \mathbf{I}_d \quad (14)$$

- $\gamma = 0$: Unregularized covariance structure.
- $\gamma = 1$: Spherical isotropic covariance matrix (Diagonal Ridge penalty).

RDA allows continuous adaptation along a 2D hyperparameter plane (α, γ) , navigating smoothly between QDA, LDA, Nearest Centroid, and Ridge classifiers.

6 Physical Analogies: Anisotropic Potential Wells and Entropic Basins

QDA's mathematical structure maps onto concepts in classical mechanics and statistical thermodynamics.

6.1 Anisotropic Harmonic Potential Wells

Consider the negative log-likelihood of an observation $\mathbf{x}$ belonging to class k as a physical potential energy $V_k(\mathbf{x})$:

$$V_k(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k) + \frac{1}{2} \log |\boldsymbol{\Sigma}_k| - \log \pi_k \quad (15)$$

In mechanics, $V_k(\mathbf{x})$ represents a d -dimensional **Anisotropic Harmonic Oscillator Potential Well**:

- The mean vector $\boldsymbol{\mu}_k$ defines the equilibrium position (the potential minimum) of state k .
- The inverse covariance matrix $\mathbf{K}_k = \boldsymbol{\Sigma}_k^{-1}$ functions as a directionally-dependent **Stiffness Tensor** (spring constants matrix).
- In LDA, all potential wells share identical stiffness tensors ($\mathbf{K}_1 = \mathbf{K}_2$). In QDA, each phase or state k possesses a unique stiffness tensor $\mathbf{K}_k$. Some directions are stiff (high precision/low variance), while others are soft (low precision/high variance).

6.2 Entropic Penalty and Basin Volume

The term $\frac{1}{2} \log |\boldsymbol{\Sigma}_k|$ measures the logarithmic volume of the state's configuration space (basin of attraction).

From Boltzmann's entropy formula $S = k_B \log \Omega$:

$$S_k \propto \frac{1}{2} \log |\boldsymbol{\Sigma}_k| \quad (16)$$

A state with a large covariance determinant $|\boldsymbol{\Sigma}_k|$ has higher internal entropy S_k , corresponding to a broader potential well. QDA balances mechanical energy ($\frac{1}{2} D_M^2$) against entropic volume ($\frac{1}{2} \log |\boldsymbol{\Sigma}_k|$) to evaluate overall thermodynamic state stability.

7 Maximum Likelihood Estimation and Numerical Computations

7.1 Parameter Estimation

Under the Gaussian assumption, parameters $\boldsymbol{\theta}_k = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ are estimated via Maximum Likelihood Estimation (MLE):

$$\hat{\pi}_k = \frac{N_k}{N} \quad (17)$$

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{i \in C_k} \mathbf{x}^{(i)} \quad (18)$$

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{N_k - 1} \sum_{i \in C_k} (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)^T \quad (19)$$

Division by $N_k - 1$ provides an unbiased sample covariance estimate.

7.2 Numerical Stability via Cholesky Decomposition

Computing $\boldsymbol{\Sigma}_k^{-1}$ and $\log |\boldsymbol{\Sigma}_k|$ directly via standard matrix inversion is computationally unstable when $\boldsymbol{\Sigma}_k$ is ill-conditioned.

Instead, we use the ****Cholesky Decomposition**** of symmetric positive-definite matrix $\boldsymbol{\Sigma}_k$:

$$\boldsymbol{\Sigma}_k = \mathbf{L}_k \mathbf{L}_k^T \quad (20)$$

where $\mathbf{L}_k$ is a lower triangular matrix.

1. Log-Determinant Computation

$$\log |\boldsymbol{\Sigma}_k| = \log |\mathbf{L}_k \mathbf{L}_k^T| = \log (|\mathbf{L}_k|^2) = 2 \log |\mathbf{L}_k| = 2 \sum_{j=1}^d \log (\mathbf{L}_k)_{jj} \quad (21)$$

This replaces direct determinant calculation with a sum over diagonal elements.

2. Quadratic Form Computation

To compute Mahalanobis distance $D_M^2 = (\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)$, we solve the lower triangular system for $\mathbf{v}$:

$$\mathbf{L}_k \mathbf{v} = (\mathbf{x} - \boldsymbol{\mu}_k) \implies \mathbf{v} = \mathbf{L}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \quad (22)$$

Then:

$$D_M^2 = \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|_2^2 \quad (23)$$

This avoids explicit matrix inversion, reducing complexity and numerical error.

8 Production-Grade Vectorized NumPy Implementation

The following Python class implements Quadratic Discriminant Analysis with Cholesky decomposition and covariance regularization.

```
1 import numpy as np
2
3 class QuadraticDiscriminantAnalysisEngine:
4     """
5     Production-grade Quadratic Discriminant Analysis (QDA) engine
6     supporting
7     heteroscedastic Gaussian modeling, Cholesky stability, and covariance
8     shrinkage.
9     """
```

```

8     def __init__(self, reg_param: float = 0.0):
9         self.reg_param = reg_param
10        self.priors_ = None           # Class priors  $\pi_k$  (K,)
11        self.means_ = None           # Class means  $\mu_k$  (K, d)
12        self.covariances_ = []       # Class covariance matrices  $\Sigma_k$  (K,
13        d, d)
14        self.rotations_ = []         # Cholesky lower triangular factors  $L_k$ 
15        self.log_det_cov_ = None     # Log determinants  $\log|\Sigma_k|$  (K,)
16        self.classes_ = None         # Unique class labels
17
18    def fit(self, X: np.ndarray, y: np.ndarray):
19        """Fits QDA parameters via Maximum Likelihood Estimation."""
20        N, d = X.shape
21        self.classes_ = np.unique(y)
22        K = len(self.classes_)
23
24        self.priors_ = np.zeros(K)
25        self.means_ = np.zeros((K, d))
26        self.log_det_cov_ = np.zeros(K)
27        self.covariances_ = []
28        self.rotations_ = []
29
30        for k_idx, k in enumerate(self.classes_):
31            X_k = X[y == k]
32            N_k = X_k.shape[0]
33
34            if N_k <= 1:
35                raise ValueError(f"Class {k} requires at least 2 samples
36                for covariance estimation.")
37
38            # 1. MLE Estimates: Priors and Means
39            self.priors_[k_idx] = N_k / N
40            self.means_[k_idx] = np.mean(X_k, axis=0)
41
42            # 2. Sample Covariance Matrix  $\Sigma_k$ 
43            X_k_centered = X_k - self.means_[k_idx]
44            Sigma_k = (X_k_centered.T @ X_k_centered) / (N_k - 1)
45
46            # 3. Regularization:  $\Sigma_k = (1 - \text{reg}) * \Sigma_k + \text{reg} * I$ 
47            if self.reg_param > 0.0:
48                gamma = np.clip(self.reg_param, 0.0, 1.0)
49                Sigma_k = (1.0 - gamma) * Sigma_k + gamma * np.eye(d)
50
51            self.covariances_.append(Sigma_k)
52
53            # 4. Numerically Stable Cholesky Factorization:  $\Sigma_k = L_k * L_k^T$ 
54            try:
55                L_k = np.linalg.cholesky(Sigma_k)
56            except np.linalg.LinAlgError:
57                # Add small jitter if matrix is nearly singular
58                Sigma_k += 1e-6 * np.eye(d)
59                L_k = np.linalg.cholesky(Sigma_k)
60
61            self.rotations_.append(L_k)
62
63            # 5. Stable Log-Determinant:  $\log|\Sigma_k| = 2 * \sum(\log(\text{diag}(L_k)))$ 
64            self.log_det_cov_[k_idx] = 2.0 * np.sum(np.log(np.diagonal(L_k)))

```

```

63
64     return self
65
66 def decision_function(self, X: np.ndarray) -> np.ndarray:
67     """Evaluates quadratic discriminant functions  $\delta_k(X)$ ."""
68     N, d = X.shape
69     K = len(self.classes_)
70     deltas = np.zeros((N, K))
71
72     for k_idx in range(K):
73         X_centered = X - self.means_[k_idx]
74         L_k = self.rotations_[k_idx]
75
76         # Solve triangular system:  $L_k * V = X\_centered^T \Rightarrow V = L_k^{-1} * X\_centered^T$ 
77         V = np.linalg.solve_triangular(L_k, X_centered.T, lower=True)
78
79         # Squared Mahalanobis Distance:  $D_M^2 = \sum(V^2, axis=0)$ 
80         mahalanobis_sq = np.sum(V**2, axis=0)
81
82         # Quadratic Discriminant:  $\delta_k(x) = -0.5 * D_M^2 - 0.5 * \log |\Sigma_k| + \log(\pi_k)$ 
83         deltas[:, k_idx] = -0.5 * mahalanobis_sq - 0.5 * self.
log_det_cov_[k_idx] + np.log(self.priors_[k_idx])
84
85     return deltas
86
87 def predict_proba(self, X: np.ndarray) -> np.ndarray:
88     """Computes posterior class probabilities  $P(Y=k|X)$  using Softmax."""
89
90     deltas = self.decision_function(X)
91     # Shift deltas for numerical stability
92     shifted_deltas = deltas - np.max(deltas, axis=1, keepdims=True)
93     exp_deltas = np.exp(shifted_deltas)
94     return exp_deltas / np.sum(exp_deltas, axis=1, keepdims=True)
95
96 def predict(self, X: np.ndarray) -> np.ndarray:
97     """Predicts class labels corresponding to maximum discriminant
function."""
98     probabilities = self.predict_proba(X)
99     max_indices = np.argmax(probabilities, axis=1)
100     return self.classes_[max_indices]
101
102 if __name__ == "__main__":
103     np.random.seed(42)
104
105     # Generate Synthetic 3-Class Heteroscedastic Dataset
106     N_per_class = 150
107
108     mu0 = np.array([-2.0, -1.0])
109     mu1 = np.array([2.0, 1.0])
110     mu2 = np.array([0.0, 3.0])
111
112     # Distinct Covariances per Class (Heteroscedastic)
113     cov0 = np.array([[1.0, 0.8], [0.8, 1.0]])
114     cov1 = np.array([[2.0, -0.5], [-0.5, 0.5]])
115     cov2 = np.array([[0.3, 0.0], [0.0, 3.0]])
116
117     X0 = np.random.multivariate_normal(mu0, cov0, N_per_class)

```



```

118 X1 = np.random.multivariate_normal(mu1, cov1, N_per_class)
119 X2 = np.random.multivariate_normal(mu2, cov2, N_per_class)
120
121 X_data = np.vstack([X0, X1, X2])
122 y_data = np.hstack([np.zeros(N_per_class), np.ones(N_per_class), 2 * np
123 .ones(N_per_class)]).astype(int)
124
125 # Train QDA Engine
126 qda = QuadraticDiscriminantAnalysisEngine(reg_param=0.01)
127 qda.fit(X_data, y_data)
128
129 # Predict
130 y_preds = qda.predict(X_data)
131 accuracy = np.mean(y_preds == y_data)
132
133 print(f"== QDA CLASSIFICATION ACCURACY: {accuracy * 100:.2f}% ==")
134 print(f"Evaluated Samples: {len(y_data)}")

```

Listing 1: Vectorized NumPy implementation of Quadratic Discriminant Analysis.